Multiclass Support Vector Machine exercise

In this exercise you will:

- implement a fully-vectorized **loss function** for the SVM
- implement the fully-vectorized expression for its **analytic gradient**
- **check your implementation** using numerical gradient
- use a validation set to **tune the learning rate and regularization** strength
- **optimize** the loss function with **SGD**
- **visualize** the final learned weights

In [1]:
```
# Run some setup code for this notebook.
import random
import numpy as np
from data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the
# notebook rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Some more magic so that the notebook will reload external python module
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in
%load_ext autoreload
%autoreload 2
```

CIFAR-10 Data Loading and Preprocessing

In [2]:
```
# Before running this block, you should use the .sh script in /dataset to
# Load the raw CIFAR-10 data.
cifar10_dir = 'datasets/cifar-10-batches-py'

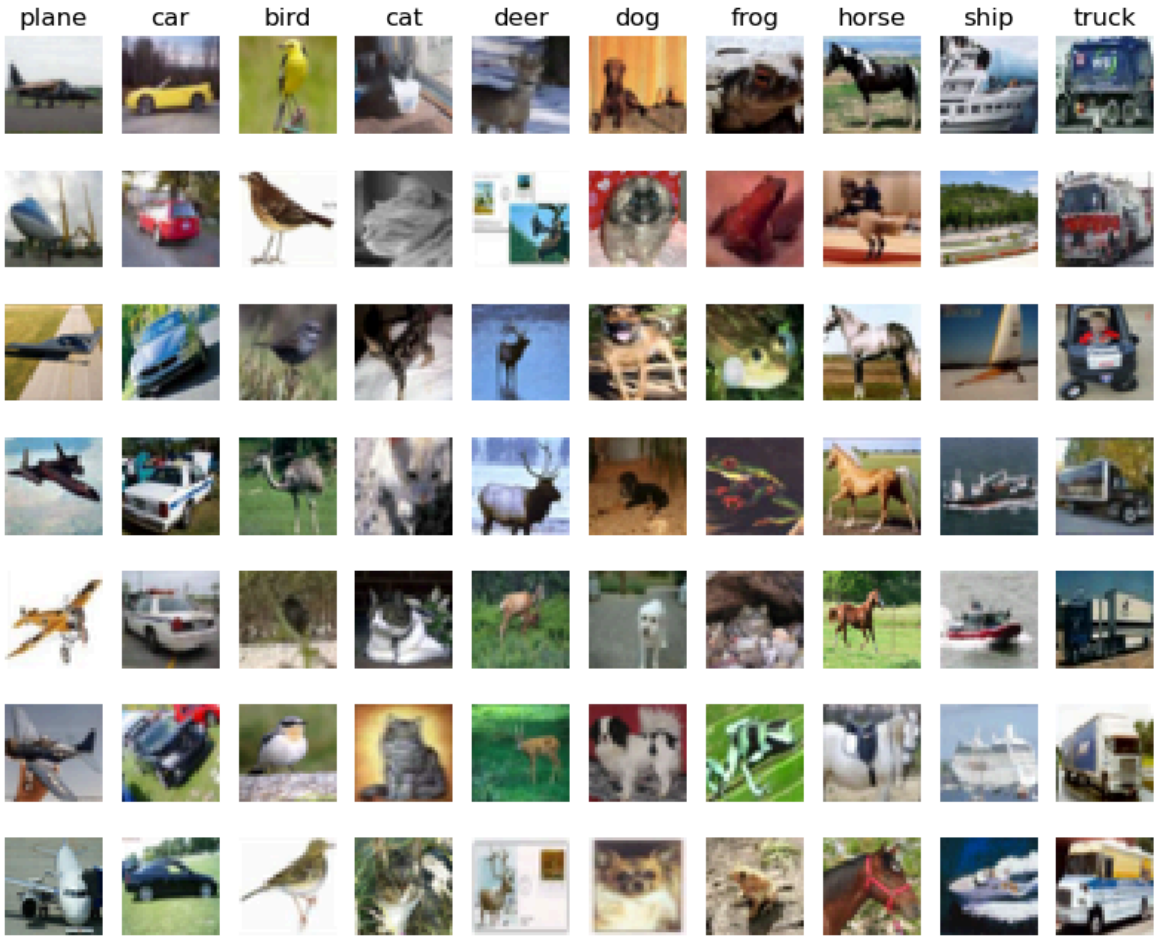
# Cleaning up variables to prevent loading data multiple times (which may
try:
    del X_train, y_train
    del X_test, y_test
    print('Clear previously loaded data.')
except:
    pass

X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Training data shape:  (50000, 32, 32, 3)
Training labels shape:  (50000,)
Test data shape:  (10000, 32, 32, 3)
Test labels shape:  (10000,)
```

In [3]:
```
# Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```



In [4]:
```
# Split the data into train, val, and test sets. In addition we will
# create a small development set as a subset of the training data;
# we can use this for development so our code runs faster.
num_training = 49000
num_validation = 1000
num_test = 1000
num_dev = 500

# Our validation set will be num_validation points from the original
```

```
# training set.
mask = range(num_training, num_training + num_validation)
X_val = X_train[mask]
y_val = y_train[mask]

# Our training set will be the first num_train points from the original
# training set.
mask = range(num_training)
X_train = X_train[mask]
y_train = y_train[mask]

# We will also make a development set, which is a small subset of
# the training set.
mask = np.random.choice(num_training, num_dev, replace=False)
X_dev = X_train[mask]
y_dev = y_train[mask]

# We use the first num_test points of the original test set as our
# test set.
mask = range(num_test)
X_test = X_test[mask]
y_test = y_test[mask]

print('Train data shape: ', X_train.shape)
print('Train labels shape: ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels shape: ', y_val.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Train data shape:  (49000, 32, 32, 3)
Train labels shape:  (49000,)
Validation data shape:  (1000, 32, 32, 3)
Validation labels shape:  (1000,)
Test data shape:  (1000, 32, 32, 3)
Test labels shape:  (1000,)
```

In [5]:
```
# Preprocessing: reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_val = np.reshape(X_val, (X_val.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
X_dev = np.reshape(X_dev, (X_dev.shape[0], -1))

# As a sanity check, print out the shapes of the data
print('Training data shape: ', X_train.shape)
print('Validation data shape: ', X_val.shape)
print('Test data shape: ', X_test.shape)
print('dev data shape: ', X_dev.shape)
```

```
Training data shape:  (49000, 3072)
Validation data shape:  (1000, 3072)
Test data shape:  (1000, 3072)
dev data shape:  (500, 3072)
```

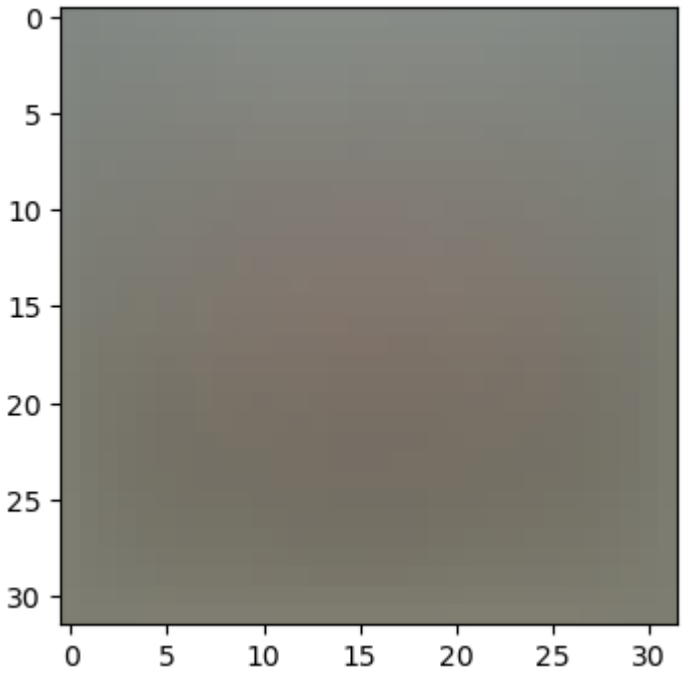
In [6]:
```
# Preprocessing: subtract the mean image
# first: compute the image mean based on the training data
mean_image = np.mean(X_train, axis=0)
print(mean_image[:10]) # print a few of the elements
plt.figure(figsize=(4,4))
plt.imshow(mean_image.reshape((32,32,3)).astype('uint8')) # visualize the
plt.show()
```

```
# second: subtract the mean image from train and test data
X_train -= mean_image
X_val -= mean_image
X_test -= mean_image
X_dev -= mean_image

# third: append the bias dimension of ones (i.e. bias trick) so that our
# only has to worry about optimizing a single weight matrix W.
X_train = np.hstack([X_train, np.ones((X_train.shape[0], 1))])
X_val = np.hstack([X_val, np.ones((X_val.shape[0], 1))])
X_test = np.hstack([X_test, np.ones((X_test.shape[0], 1))])
X_dev = np.hstack([X_dev, np.ones((X_dev.shape[0], 1))])

print(X_train.shape, X_val.shape, X_test.shape, X_dev.shape)
```

```
[130.64189796 135.98173469 132.47391837 130.05569388 135.34804082
 131.75402041 130.96055102 136.14328571 132.47636735 131.48467347]
```




```
(49000, 3073) (1000, 3073) (1000, 3073) (500, 3073)
```

SVM Classifier

Your code for this section will all be written inside `classifiers/linear_svm.py`.

As you can see, we have prefilled the function `svm_loss_naive` which uses for loops to evaluate the multiclass SVM loss function.

In [7]:
```
# Evaluate the naive implementation of the loss we provided for you:
from classifiers.linear_svm import svm_loss_naive
import time

# generate a random SVM weight matrix of small numbers
W = np.random.randn(3073, 10) * 0.0001

# we should download the 'past' module by 'pip install future' ????!!!!
```

```
loss, grad = svm_loss_naive(W, X_dev, y_dev, 0.000005)
print('loss: %f' % (loss, ))
```

```
loss: 9.067567
```

The `grad` returned from the function above is right now all zero. Derive and implement the gradient for the SVM cost function and implement it inline inside the function `svm_loss_naive`. You will find it helpful to interleave your new code inside the existing function.

To check that you have correctly implemented the gradient correctly, you can numerically estimate the gradient of the loss function and compare the numeric estimate to the gradient that you computed. We have provided code that does this for you:

In [8]:
```
# Once you've implemented the gradient, recompute it with the code below
# and gradient check it with the function we provided for you

# Compute the loss and its gradient at W.
loss, grad = svm_loss_naive(W, X_dev, y_dev, 0.0)

# Numerically compute the gradient along several randomly chosen dimensio
# compare them with your analytically computed gradient. The numbers shou
# almost exactly along all dimensions.
from gradient_check import grad_check_sparse
f = lambda w: svm_loss_naive(w, X_dev, y_dev, 0.0)[0]
grad_numerical = grad_check_sparse(f, W, grad)

# do the gradient check once again with regularization turned on
# you didn't forget the regularization gradient did you?

print("=============== with regularization turned on ===============")
loss, grad = svm_loss_naive(W, X_dev, y_dev, 5e1)
f = lambda w: svm_loss_naive(w, X_dev, y_dev, 5e1)[0]
grad_numerical = grad_check_sparse(f, W, grad)
```

```
numerical: -5.516121 analytic: -5.516121, relative error: 1.002423e-11
numerical: 11.657803 analytic: 11.657803, relative error: 2.184051e-11
numerical: 8.510624 analytic: 8.510624, relative error: 1.944457e-11
numerical: 22.015386 analytic: 22.015386, relative error: 3.656415e-12
numerical: -47.357089 analytic: -47.357089, relative error: 3.893146e-12
numerical: 0.930269 analytic: 0.930269, relative error: 8.973948e-11
numerical: 8.215102 analytic: 8.215102, relative error: 7.689270e-12
numerical: 3.594295 analytic: 3.594295, relative error: 1.331832e-10
numerical: -5.305248 analytic: -5.305248, relative error: 6.257046e-11
numerical: -10.941006 analytic: -10.941006, relative error: 8.560311e-13
=============== with regularization turned on ===============
numerical: 3.016471 analytic: 3.016471, relative error: 1.697201e-10
numerical: -14.650399 analytic: -14.650399, relative error: 1.510220e-11
numerical: -11.175482 analytic: -11.175482, relative error: 5.102335e-14
numerical: 22.441507 analytic: 22.441507, relative error: 4.830195e-12
numerical: 0.296994 analytic: 0.296994, relative error: 5.226084e-10
numerical: -7.864008 analytic: -7.864008, relative error: 1.576182e-11
numerical: -13.139648 analytic: -13.139648, relative error: 9.016469e-12
numerical: 0.912507 analytic: 0.912507, relative error: 5.581908e-11
numerical: 14.084841 analytic: 14.084841, relative error: 2.920491e-11
numerical: 8.758743 analytic: 8.758743, relative error: 1.774097e-11
```

Inline Question 1

It is possible that once in a while a dimension in the gradcheck will not match exactly. What could such a discrepancy be caused by? Is it a reason for concern? What is a simple example in one dimension where a gradient check could fail? How would change the margin affect of the frequency of this happening? *Hint: the SVM loss function is not strictly speaking differentiable*

Your Answer :

The discrepancy in the gradient check can be caused by:

1. Non-differentiability at certain points: The SVM loss function includes a loss term $max(0, s_j - s_{y_i} + \Delta)$, at $s_j - s_{y_i} + \Delta = 0$ point, the function is not differentiable, and numerical gradient approximation may differ from analytical gradients.

2. Numerical precision issues: Finite difference approximations involve subtracting similar numbers, which can lead to floating-point errors.

This is generally not a major concern unless the discrepancies are large or occur frequently.

A simple one-dimensional example: Consider a simplified hinge loss $L(w) = max(0, 1 - w)$. If we compute the analytical gradient at $w = 1$, we get 0 (from the right) or -1 (from the left), but numerical methods might approximate something in between.

Regarding margin Δ: Increasing the margin would shift the non-differentiable points in the loss function. With a larger margin, more training examples would fall into the region where the hinge loss is active (non-zero), increasing the likelihood of encountering these kinks during gradient checks. Therefore, a larger margin would likely increase the frequency of gradient check discrepancies.

In [9]:
```
# Next implement the function svm_loss_vectorized; for now only compute t
# we will implement the gradient in a moment.
tic = time.time()
loss_naive, grad_naive = svm_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss: %e computed in %fs' % (loss_naive, toc - tic))

from classifiers.linear_svm import svm_loss_vectorized
tic = time.time()
loss_vectorized, _ = svm_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss: %e computed in %fs' % (loss_vectorized, toc - tic

# The losses should match but your vectorized implementation should be mu
print('difference: %f' % (loss_naive - loss_vectorized))
```

```
Naive loss: 9.067567e+00 computed in 0.714318s
Vectorized loss: 9.067567e+00 computed in 0.066618s
difference: 0.000000
```

In [10]:
```
# Complete the implementation of svm_loss_vectorized, and compute the gra
# of the loss function in a vectorized way.

# The naive implementation and the vectorized implementation should match
# the vectorized version should still be much faster.
tic = time.time()
_, grad_naive = svm_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss and gradient: computed in %fs' % (toc - tic))

tic = time.time()
_, grad_vectorized = svm_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss and gradient: computed in %fs' % (toc - tic))

# The loss is a single number, so it is easy to compare the values comput
# by the two implementations. The gradient on the other hand is a matrix,
# we use the Frobenius norm to compare them.
difference = np.linalg.norm(grad_naive - grad_vectorized, ord='fro')
print('difference: %f' % difference)
```

```
Naive loss and gradient: computed in 0.259757s
Vectorized loss and gradient: computed in 0.002198s
difference: 0.000000
```

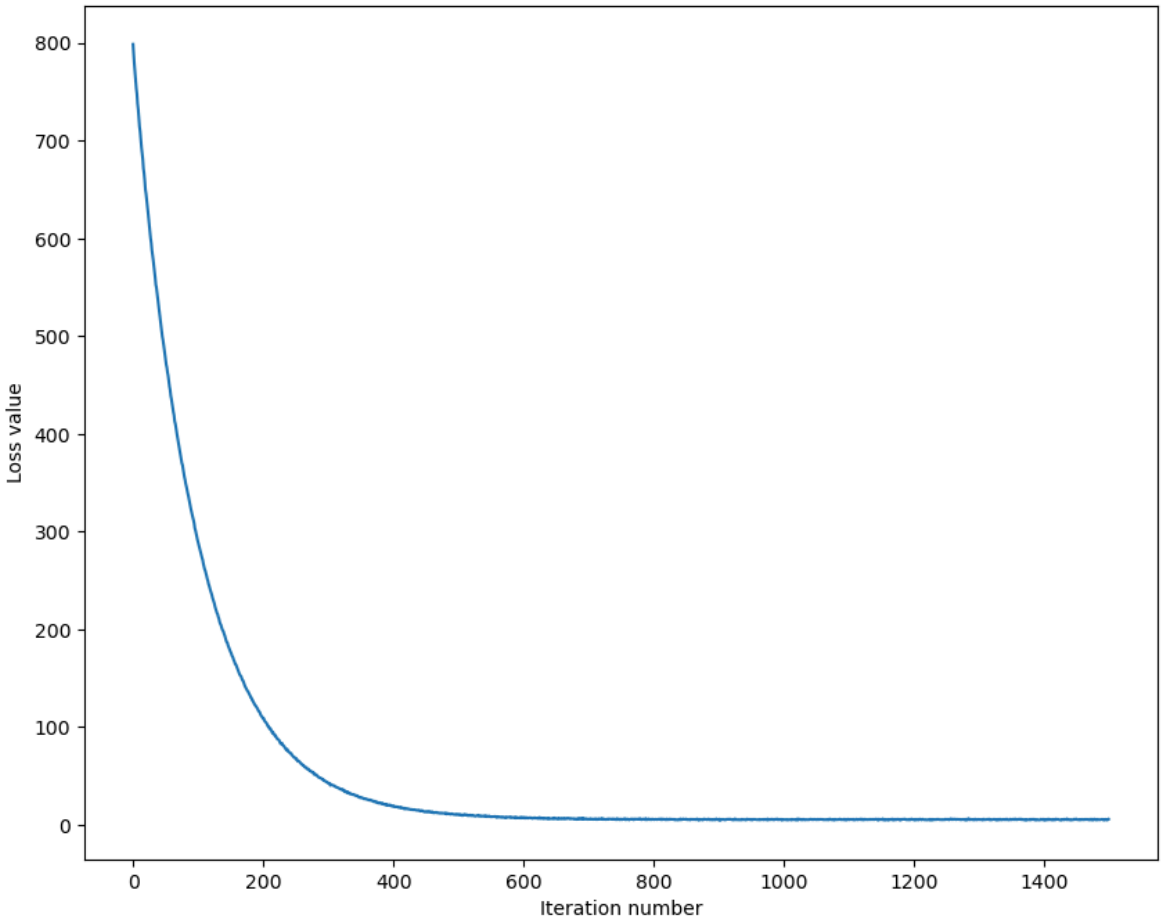
Stochastic Gradient Descent

We now have vectorized and efficient expressions for the loss, the gradient and our gradient matches the numerical gradient. We are therefore ready to do SGD to minimize the loss. Your code for this part will be written inside `classifiers/linear_classifier.py`.

In [11]:
```
# In the file linear_classifier.py, implement SGD in the function
# LinearClassifier.train() and then run it with the code below.
from classifiers import LinearSVM
svm = LinearSVM()
tic = time.time()
loss_hist = svm.train(X_train, y_train, learning_rate=1e-7, reg=2.5e4,
                      num_iters=1500, verbose=True)
toc = time.time()
print('That took %fs' % (toc - tic))
```

```
iteration 0 / 1500: loss 798.531781
iteration 100 / 1500: loss 288.921317
iteration 200 / 1500: loss 108.609752
iteration 300 / 1500: loss 42.519464
iteration 400 / 1500: loss 18.637202
iteration 500 / 1500: loss 10.588823
iteration 600 / 1500: loss 7.005050
iteration 700 / 1500: loss 5.514410
iteration 800 / 1500: loss 5.276365
iteration 900 / 1500: loss 5.036449
iteration 1000 / 1500: loss 5.423853
iteration 1100 / 1500: loss 5.242792
iteration 1200 / 1500: loss 5.111350
iteration 1300 / 1500: loss 5.244036
iteration 1400 / 1500: loss 4.883692
That took 11.058791s
```

In [12]:
```
# A useful debugging strategy is to plot the loss as a function of
# iteration number:
plt.plot(loss_hist)
plt.xlabel('Iteration number')
plt.ylabel('Loss value')
plt.show()
```



In [13]:
```
# Write the LinearSVM.predict function and evaluate the performance on bo
# training and validation set
y_train_pred = svm.predict(X_train)
print('training accuracy: %f' % (np.mean(y_train == y_train_pred), ))
y_val_pred = svm.predict(X_val)
print('validation accuracy: %f' % (np.mean(y_val == y_val_pred), ))
```

```
training accuracy: 0.363510
validation accuracy: 0.378000
```

In [14]:
```
# Use the validation set to tune hyperparameters (regularization strength
# learning rate). You should experiment with different ranges for the lea
# rates and regularization strengths; if you are careful you should be ab
# get a classification accuracy of about 0.39 on the validation set.

# Note: you may see runtime/overflow warnings during hyper-parameter sear
# This may be caused by extreme values, and is not a bug.

# results is dictionary mapping tuples of the form
# (learning_rate, regularization_strength) to tuples of the form
# (training_accuracy, validation_accuracy). The accuracy is simply the fr
# of data points that are correctly classified.
results = {}
best_val = -1   # The highest validation accuracy that we have seen so fa
```

```
best_svm = None # The LinearSVM object that achieved the highest validati

################################################################################
# TODO:
# Write code that chooses the best hyperparameters by tuning on the valid
# set. For each combination of hyperparameters, train a linear SVM on the
# training set, compute its accuracy on the training and validation sets,
# store these numbers in the results dictionary. In addition, store the b
# validation accuracy in best_val and the LinearSVM object that achieves
# accuracy in best_svm.
#
# Hint: You should use a small value for num_iters as you develop your
# validation code so that the SVMs don't take much time to train; once yo
# confident that your validation code works, you should rerun the validat
# code with a larger value for num_iters.
################################################################################

# Provided as a reference. You can change these hyperparameters if you li

delta_lr = 5e-8
delta_reg = 5e3
learning_rates = [lr * delta_lr for lr in range(1, 10)]
regularization_strengths = [reg * delta_reg for reg in range(1, 10)]


# *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****

# Set a fixed number of iterations for training
num_iters = 1500  # Start with a smaller number like 100 for testing, the

# Iterate over all combinations of learning rates and regularization stre
for lr in learning_rates:
    for reg in regularization_strengths:
        # Create and train a new SVM model
        svm = LinearSVM()
        svm.train(X_train, y_train, learning_rate=lr, reg=reg, num_iters=

        # Evaluate on training set
        y_train_pred = svm.predict(X_train)
        train_accuracy = np.mean(y_train == y_train_pred)

        # Evaluate on validation set
        y_val_pred = svm.predict(X_val)
        val_accuracy = np.mean(y_val == y_val_pred)

        # Store results
        results[(lr, reg)] = (train_accuracy, val_accuracy)

        # Update best model if necessary
        if val_accuracy > best_val:
            best_val = val_accuracy
            best_svm = svm

        # Print progress
        print(f'lr {lr:.2e}, reg {reg:.2e}: train accuracy: {train_accura

# *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****

# Print out results.
for lr, reg in sorted(results):
```

```
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
                lr, reg, train_accuracy, val_accuracy))

print('best validation accuracy achieved during cross-validation: %f' % b
```

```
lr 5.00e-08, reg 5.00e+03: train accuracy: 0.3231, val accuracy: 0.3550
lr 5.00e-08, reg 1.00e+04: train accuracy: 0.3552, val accuracy: 0.3610
lr 5.00e-08, reg 1.50e+04: train accuracy: 0.3710, val accuracy: 0.3780
lr 5.00e-08, reg 2.00e+04: train accuracy: 0.3746, val accuracy: 0.3790
lr 5.00e-08, reg 2.50e+04: train accuracy: 0.3736, val accuracy: 0.3840
lr 5.00e-08, reg 3.00e+04: train accuracy: 0.3707, val accuracy: 0.3850
lr 5.00e-08, reg 3.50e+04: train accuracy: 0.3629, val accuracy: 0.3790
lr 5.00e-08, reg 4.00e+04: train accuracy: 0.3651, val accuracy: 0.3750
lr 5.00e-08, reg 4.50e+04: train accuracy: 0.3617, val accuracy: 0.3770
lr 1.00e-07, reg 5.00e+03: train accuracy: 0.3713, val accuracy: 0.3830
lr 1.00e-07, reg 1.00e+04: train accuracy: 0.3828, val accuracy: 0.3900
lr 1.00e-07, reg 1.50e+04: train accuracy: 0.3803, val accuracy: 0.4000
lr 1.00e-07, reg 2.00e+04: train accuracy: 0.3687, val accuracy: 0.3840
lr 1.00e-07, reg 2.50e+04: train accuracy: 0.3751, val accuracy: 0.3910
lr 1.00e-07, reg 3.00e+04: train accuracy: 0.3662, val accuracy: 0.3760
lr 1.00e-07, reg 3.50e+04: train accuracy: 0.3573, val accuracy: 0.3620
lr 1.00e-07, reg 4.00e+04: train accuracy: 0.3648, val accuracy: 0.3730
lr 1.00e-07, reg 4.50e+04: train accuracy: 0.3577, val accuracy: 0.3720
lr 1.50e-07, reg 5.00e+03: train accuracy: 0.3866, val accuracy: 0.3890
lr 1.50e-07, reg 1.00e+04: train accuracy: 0.3811, val accuracy: 0.3850
lr 1.50e-07, reg 1.50e+04: train accuracy: 0.3726, val accuracy: 0.3950
lr 1.50e-07, reg 2.00e+04: train accuracy: 0.3691, val accuracy: 0.3700
lr 1.50e-07, reg 2.50e+04: train accuracy: 0.3672, val accuracy: 0.3820
lr 1.50e-07, reg 3.00e+04: train accuracy: 0.3582, val accuracy: 0.3710
lr 1.50e-07, reg 3.50e+04: train accuracy: 0.3546, val accuracy: 0.3700
lr 1.50e-07, reg 4.00e+04: train accuracy: 0.3623, val accuracy: 0.3690
lr 1.50e-07, reg 4.50e+04: train accuracy: 0.3584, val accuracy: 0.3740
lr 2.00e-07, reg 5.00e+03: train accuracy: 0.3862, val accuracy: 0.3870
lr 2.00e-07, reg 1.00e+04: train accuracy: 0.3764, val accuracy: 0.3710
lr 2.00e-07, reg 1.50e+04: train accuracy: 0.3711, val accuracy: 0.3840
lr 2.00e-07, reg 2.00e+04: train accuracy: 0.3508, val accuracy: 0.3760
lr 2.00e-07, reg 2.50e+04: train accuracy: 0.3531, val accuracy: 0.3460
lr 2.00e-07, reg 3.00e+04: train accuracy: 0.3579, val accuracy: 0.3540
lr 2.00e-07, reg 3.50e+04: train accuracy: 0.3445, val accuracy: 0.3570
lr 2.00e-07, reg 4.00e+04: train accuracy: 0.3491, val accuracy: 0.3700
lr 2.00e-07, reg 4.50e+04: train accuracy: 0.3398, val accuracy: 0.3520
lr 2.50e-07, reg 5.00e+03: train accuracy: 0.3778, val accuracy: 0.3770
lr 2.50e-07, reg 1.00e+04: train accuracy: 0.3806, val accuracy: 0.3950
lr 2.50e-07, reg 1.50e+04: train accuracy: 0.3709, val accuracy: 0.3860
lr 2.50e-07, reg 2.00e+04: train accuracy: 0.3554, val accuracy: 0.3650
lr 2.50e-07, reg 2.50e+04: train accuracy: 0.3656, val accuracy: 0.3740
lr 2.50e-07, reg 3.00e+04: train accuracy: 0.3525, val accuracy: 0.3680
lr 2.50e-07, reg 3.50e+04: train accuracy: 0.3326, val accuracy: 0.3510
lr 2.50e-07, reg 4.00e+04: train accuracy: 0.3393, val accuracy: 0.3380
lr 2.50e-07, reg 4.50e+04: train accuracy: 0.3492, val accuracy: 0.3630
lr 3.00e-07, reg 5.00e+03: train accuracy: 0.3791, val accuracy: 0.3670
lr 3.00e-07, reg 1.00e+04: train accuracy: 0.3748, val accuracy: 0.3690
lr 3.00e-07, reg 1.50e+04: train accuracy: 0.3622, val accuracy: 0.3710
lr 3.00e-07, reg 2.00e+04: train accuracy: 0.3651, val accuracy: 0.3650
lr 3.00e-07, reg 2.50e+04: train accuracy: 0.3550, val accuracy: 0.3730
lr 3.00e-07, reg 3.00e+04: train accuracy: 0.3464, val accuracy: 0.3690
lr 3.00e-07, reg 3.50e+04: train accuracy: 0.3387, val accuracy: 0.3340
lr 3.00e-07, reg 4.00e+04: train accuracy: 0.3420, val accuracy: 0.3500
lr 3.00e-07, reg 4.50e+04: train accuracy: 0.3414, val accuracy: 0.3570
lr 3.50e-07, reg 5.00e+03: train accuracy: 0.3668, val accuracy: 0.3650
lr 3.50e-07, reg 1.00e+04: train accuracy: 0.3557, val accuracy: 0.3640
lr 3.50e-07, reg 1.50e+04: train accuracy: 0.3409, val accuracy: 0.3560
lr 3.50e-07, reg 2.00e+04: train accuracy: 0.3558, val accuracy: 0.3690
lr 3.50e-07, reg 2.50e+04: train accuracy: 0.3541, val accuracy: 0.3400
lr 3.50e-07, reg 3.00e+04: train accuracy: 0.3422, val accuracy: 0.3700
```

```
lr 3.50e-07, reg 3.50e+04: train accuracy: 0.3515, val accuracy: 0.3580
lr 3.50e-07, reg 4.00e+04: train accuracy: 0.3362, val accuracy: 0.3450
lr 3.50e-07, reg 4.50e+04: train accuracy: 0.3314, val accuracy: 0.3430
lr 4.00e-07, reg 5.00e+03: train accuracy: 0.3763, val accuracy: 0.3780
lr 4.00e-07, reg 1.00e+04: train accuracy: 0.3540, val accuracy: 0.3480
lr 4.00e-07, reg 1.50e+04: train accuracy: 0.3476, val accuracy: 0.3560
lr 4.00e-07, reg 2.00e+04: train accuracy: 0.3652, val accuracy: 0.3660
lr 4.00e-07, reg 2.50e+04: train accuracy: 0.3294, val accuracy: 0.3420
lr 4.00e-07, reg 3.00e+04: train accuracy: 0.3487, val accuracy: 0.3710
lr 4.00e-07, reg 3.50e+04: train accuracy: 0.3426, val accuracy: 0.3370
lr 4.00e-07, reg 4.00e+04: train accuracy: 0.3231, val accuracy: 0.3320
lr 4.00e-07, reg 4.50e+04: train accuracy: 0.3230, val accuracy: 0.3240
lr 4.50e-07, reg 5.00e+03: train accuracy: 0.3548, val accuracy: 0.3510
lr 4.50e-07, reg 1.00e+04: train accuracy: 0.3502, val accuracy: 0.3620
lr 4.50e-07, reg 1.50e+04: train accuracy: 0.3519, val accuracy: 0.3580
lr 4.50e-07, reg 2.00e+04: train accuracy: 0.3514, val accuracy: 0.3550
lr 4.50e-07, reg 2.50e+04: train accuracy: 0.3438, val accuracy: 0.3500
lr 4.50e-07, reg 3.00e+04: train accuracy: 0.3406, val accuracy: 0.3550
lr 4.50e-07, reg 3.50e+04: train accuracy: 0.3303, val accuracy: 0.3280
lr 4.50e-07, reg 4.00e+04: train accuracy: 0.3160, val accuracy: 0.3410
lr 4.50e-07, reg 4.50e+04: train accuracy: 0.2980, val accuracy: 0.3220
lr 5.000000e-08 reg 5.000000e+03 train accuracy: 0.323102 val accuracy: 0.
355000
lr 5.000000e-08 reg 1.000000e+04 train accuracy: 0.355184 val accuracy: 0.
361000
lr 5.000000e-08 reg 1.500000e+04 train accuracy: 0.371000 val accuracy: 0.
378000
lr 5.000000e-08 reg 2.000000e+04 train accuracy: 0.374592 val accuracy: 0.
379000
lr 5.000000e-08 reg 2.500000e+04 train accuracy: 0.373612 val accuracy: 0.
384000
lr 5.000000e-08 reg 3.000000e+04 train accuracy: 0.370673 val accuracy: 0.
385000
lr 5.000000e-08 reg 3.500000e+04 train accuracy: 0.362918 val accuracy: 0.
379000
lr 5.000000e-08 reg 4.000000e+04 train accuracy: 0.365122 val accuracy: 0.
375000
lr 5.000000e-08 reg 4.500000e+04 train accuracy: 0.361673 val accuracy: 0.
377000
lr 1.000000e-07 reg 5.000000e+03 train accuracy: 0.371327 val accuracy: 0.
383000
lr 1.000000e-07 reg 1.000000e+04 train accuracy: 0.382755 val accuracy: 0.
390000
lr 1.000000e-07 reg 1.500000e+04 train accuracy: 0.380265 val accuracy: 0.
400000
lr 1.000000e-07 reg 2.000000e+04 train accuracy: 0.368653 val accuracy: 0.
384000
lr 1.000000e-07 reg 2.500000e+04 train accuracy: 0.375102 val accuracy: 0.
391000
lr 1.000000e-07 reg 3.000000e+04 train accuracy: 0.366184 val accuracy: 0.
376000
lr 1.000000e-07 reg 3.500000e+04 train accuracy: 0.357265 val accuracy: 0.
362000
lr 1.000000e-07 reg 4.000000e+04 train accuracy: 0.364816 val accuracy: 0.
373000
lr 1.000000e-07 reg 4.500000e+04 train accuracy: 0.357735 val accuracy: 0.
372000
lr 1.500000e-07 reg 5.000000e+03 train accuracy: 0.386551 val accuracy: 0.
389000
lr 1.500000e-07 reg 1.000000e+04 train accuracy: 0.381061 val accuracy: 0.
```

```
385000
lr 1.500000e-07 reg 1.500000e+04 train accuracy: 0.372633 val accuracy: 0.
395000
lr 1.500000e-07 reg 2.000000e+04 train accuracy: 0.369082 val accuracy: 0.
370000
lr 1.500000e-07 reg 2.500000e+04 train accuracy: 0.367163 val accuracy: 0.
382000
lr 1.500000e-07 reg 3.000000e+04 train accuracy: 0.358184 val accuracy: 0.
371000
lr 1.500000e-07 reg 3.500000e+04 train accuracy: 0.354633 val accuracy: 0.
370000
lr 1.500000e-07 reg 4.000000e+04 train accuracy: 0.362286 val accuracy: 0.
369000
lr 1.500000e-07 reg 4.500000e+04 train accuracy: 0.358449 val accuracy: 0.
374000
lr 2.000000e-07 reg 5.000000e+03 train accuracy: 0.386163 val accuracy: 0.
387000
lr 2.000000e-07 reg 1.000000e+04 train accuracy: 0.376449 val accuracy: 0.
371000
lr 2.000000e-07 reg 1.500000e+04 train accuracy: 0.371061 val accuracy: 0.
384000
lr 2.000000e-07 reg 2.000000e+04 train accuracy: 0.350816 val accuracy: 0.
376000
lr 2.000000e-07 reg 2.500000e+04 train accuracy: 0.353122 val accuracy: 0.
346000
lr 2.000000e-07 reg 3.000000e+04 train accuracy: 0.357878 val accuracy: 0.
354000
lr 2.000000e-07 reg 3.500000e+04 train accuracy: 0.344510 val accuracy: 0.
357000
lr 2.000000e-07 reg 4.000000e+04 train accuracy: 0.349102 val accuracy: 0.
370000
lr 2.000000e-07 reg 4.500000e+04 train accuracy: 0.339816 val accuracy: 0.
352000
lr 2.500000e-07 reg 5.000000e+03 train accuracy: 0.377816 val accuracy: 0.
377000
lr 2.500000e-07 reg 1.000000e+04 train accuracy: 0.380551 val accuracy: 0.
395000
lr 2.500000e-07 reg 1.500000e+04 train accuracy: 0.370918 val accuracy: 0.
386000
lr 2.500000e-07 reg 2.000000e+04 train accuracy: 0.355429 val accuracy: 0.
365000
lr 2.500000e-07 reg 2.500000e+04 train accuracy: 0.365633 val accuracy: 0.
374000
lr 2.500000e-07 reg 3.000000e+04 train accuracy: 0.352510 val accuracy: 0.
368000
lr 2.500000e-07 reg 3.500000e+04 train accuracy: 0.332571 val accuracy: 0.
351000
lr 2.500000e-07 reg 4.000000e+04 train accuracy: 0.339327 val accuracy: 0.
338000
lr 2.500000e-07 reg 4.500000e+04 train accuracy: 0.349224 val accuracy: 0.
363000
lr 3.000000e-07 reg 5.000000e+03 train accuracy: 0.379061 val accuracy: 0.
367000
lr 3.000000e-07 reg 1.000000e+04 train accuracy: 0.374816 val accuracy: 0.
369000
lr 3.000000e-07 reg 1.500000e+04 train accuracy: 0.362245 val accuracy: 0.
371000
lr 3.000000e-07 reg 2.000000e+04 train accuracy: 0.365082 val accuracy: 0.
365000
lr 3.000000e-07 reg 2.500000e+04 train accuracy: 0.355020 val accuracy: 0.
```

```
373000
lr 3.000000e-07 reg 3.000000e+04 train accuracy: 0.346388 val accuracy: 0.
369000
lr 3.000000e-07 reg 3.500000e+04 train accuracy: 0.338694 val accuracy: 0.
334000
lr 3.000000e-07 reg 4.000000e+04 train accuracy: 0.342000 val accuracy: 0.
350000
lr 3.000000e-07 reg 4.500000e+04 train accuracy: 0.341367 val accuracy: 0.
357000
lr 3.500000e-07 reg 5.000000e+03 train accuracy: 0.366837 val accuracy: 0.
365000
lr 3.500000e-07 reg 1.000000e+04 train accuracy: 0.355714 val accuracy: 0.
364000
lr 3.500000e-07 reg 1.500000e+04 train accuracy: 0.340939 val accuracy: 0.
356000
lr 3.500000e-07 reg 2.000000e+04 train accuracy: 0.355816 val accuracy: 0.
369000
lr 3.500000e-07 reg 2.500000e+04 train accuracy: 0.354061 val accuracy: 0.
340000
lr 3.500000e-07 reg 3.000000e+04 train accuracy: 0.342245 val accuracy: 0.
370000
lr 3.500000e-07 reg 3.500000e+04 train accuracy: 0.351531 val accuracy: 0.
358000
lr 3.500000e-07 reg 4.000000e+04 train accuracy: 0.336245 val accuracy: 0.
345000
lr 3.500000e-07 reg 4.500000e+04 train accuracy: 0.331429 val accuracy: 0.
343000
lr 4.000000e-07 reg 5.000000e+03 train accuracy: 0.376327 val accuracy: 0.
378000
lr 4.000000e-07 reg 1.000000e+04 train accuracy: 0.354041 val accuracy: 0.
348000
lr 4.000000e-07 reg 1.500000e+04 train accuracy: 0.347551 val accuracy: 0.
356000
lr 4.000000e-07 reg 2.000000e+04 train accuracy: 0.365163 val accuracy: 0.
366000
lr 4.000000e-07 reg 2.500000e+04 train accuracy: 0.329367 val accuracy: 0.
342000
lr 4.000000e-07 reg 3.000000e+04 train accuracy: 0.348694 val accuracy: 0.
371000
lr 4.000000e-07 reg 3.500000e+04 train accuracy: 0.342612 val accuracy: 0.
337000
lr 4.000000e-07 reg 4.000000e+04 train accuracy: 0.323122 val accuracy: 0.
332000
lr 4.000000e-07 reg 4.500000e+04 train accuracy: 0.323020 val accuracy: 0.
324000
lr 4.500000e-07 reg 5.000000e+03 train accuracy: 0.354837 val accuracy: 0.
351000
lr 4.500000e-07 reg 1.000000e+04 train accuracy: 0.350163 val accuracy: 0.
362000
lr 4.500000e-07 reg 1.500000e+04 train accuracy: 0.351898 val accuracy: 0.
358000
lr 4.500000e-07 reg 2.000000e+04 train accuracy: 0.351408 val accuracy: 0.
355000
lr 4.500000e-07 reg 2.500000e+04 train accuracy: 0.343755 val accuracy: 0.
350000
lr 4.500000e-07 reg 3.000000e+04 train accuracy: 0.340571 val accuracy: 0.
355000
lr 4.500000e-07 reg 3.500000e+04 train accuracy: 0.330347 val accuracy: 0.
328000
lr 4.500000e-07 reg 4.000000e+04 train accuracy: 0.315980 val accuracy: 0.
```

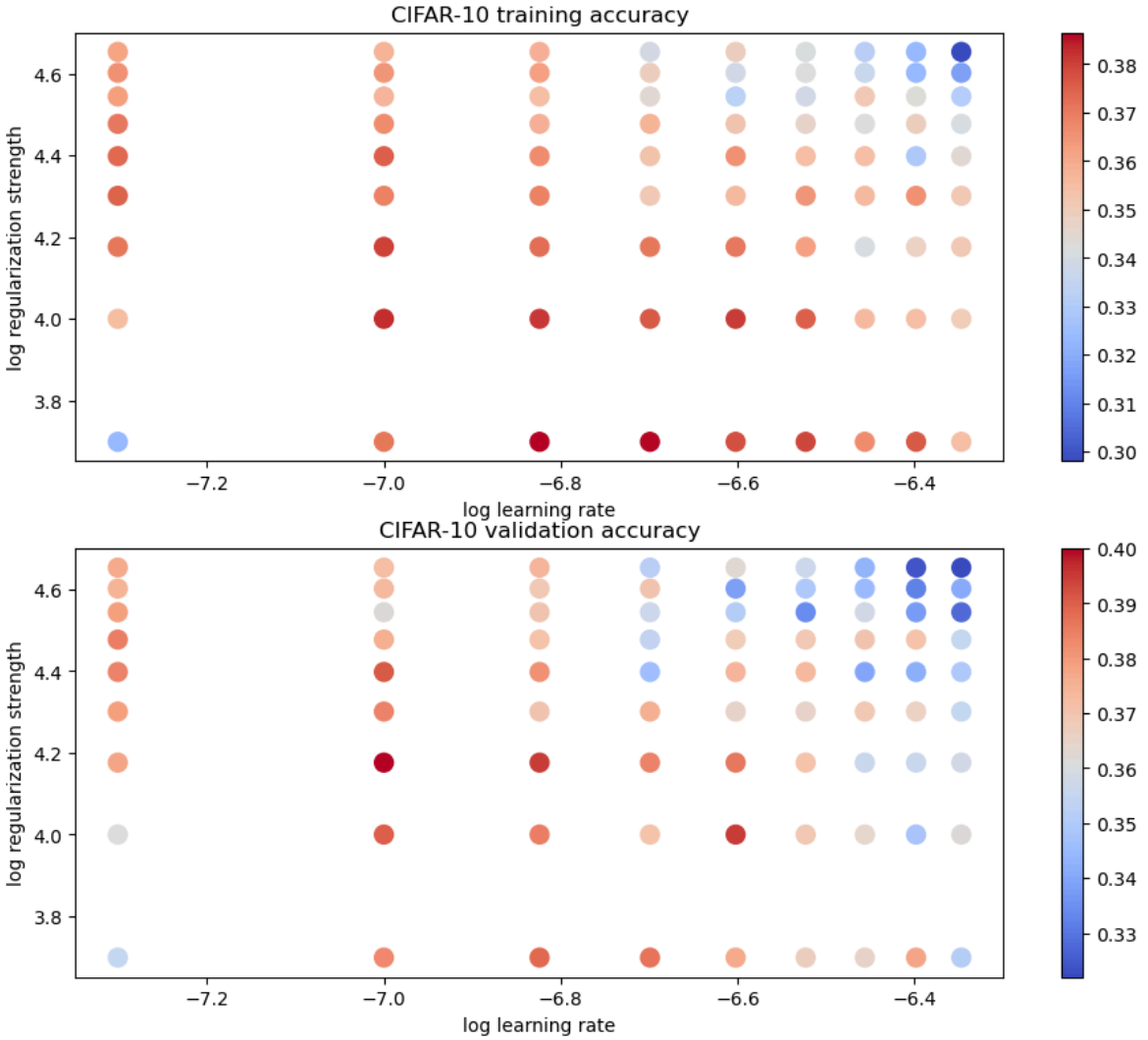
```
341000
lr 4.500000e-07 reg 4.500000e+04 train accuracy: 0.297980 val accuracy: 0.
322000
best validation accuracy achieved during cross-validation: 0.400000
```

In [15]:
```python
# Visualize the cross-validation results
import math

x_scatter = [math.log10(x[0]) for x in results]
y_scatter = [math.log10(x[1]) for x in results]

# plot training accuracy
marker_size = 100
colors = [results[x][0] for x in results]
plt.subplot(2, 1, 1)
plt.tight_layout(pad=3)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.cool
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 training accuracy')

# plot validation accuracy
colors = [results[x][1] for x in results] # default size of markers is 20
plt.subplot(2, 1, 2)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.cool
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 validation accuracy')
plt.show()
```



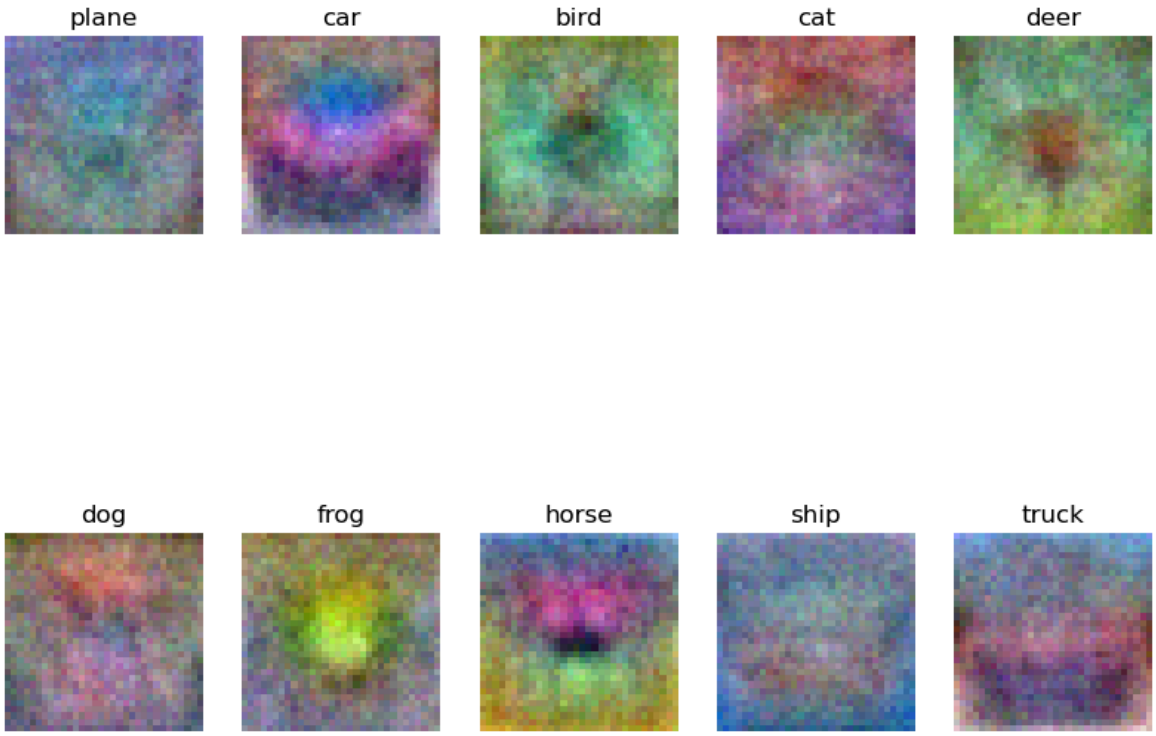
In [16]:
```
# Evaluate the best svm on test set
y_test_pred = best_svm.predict(X_test)
test_accuracy = np.mean(y_test == y_test_pred)
print('linear SVM on raw pixels final test set accuracy: %f' % test_accur
```

```
linear SVM on raw pixels final test set accuracy: 0.381000
```

In [19]:
```
# Visualize the learned weights for each class.
# Depending on your choice of learning rate and regularization strength,
# or may not be nice to look at.
w = best_svm.W[:-1,:] # strip out the bias
w = w.reshape(32, 32, 3, 10)
w_min, w_max = np.min(w), np.max(w)
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',
for i in range(10):
    plt.subplot(2, 5, i + 1)

    # Rescale the weights to be between 0 and 255
    wimg = 255.0 * (w[:, :, :, i].squeeze() - w_min) / (w_max - w_min)
    plt.imshow(wimg.astype('uint8'))
    plt.axis('off')
    plt.title(classes[i])

plt.show()
```



Inline question 2

Describe what your visualized SVM weights look like, and offer a brief explanation for why they look they way that they do.

Your Answer :

The visualized SVM weights appear as template-like images that resemble the average shape of each digit class in the CIFAR-10 dataset.

This is because the SVM classifier learns to separate different classes by finding hyperplanes that maximize the margin between them. The weights for each class represent the direction and magnitude of the decision boundary for that class. The weights are shaped like the average of the pixels in each class, as the SVM tries to minimize the loss function by adjusting the weights to best separate the classes. The resulting images show the learned features that are most discriminative for each class, which is why they resemble the average shape of the digits. The weights are also influenced by the regularization term, which encourages the model to find a simpler decision boundary that generalizes well to unseen data. This can lead to smoother and more interpretable weight visualizations.